

CSCE 605 COMPILER DESIGN · MODULE 02 · LECTURE NOTES

Lexical Analysis

Characters to tokens, by a machine you can derive.

MODULE OUTCOMES

- Specify tokens with regular expressions.
- Convert a regular expression to an NFA and then to a DFA.
- Explain maximal munch and the rule-priority convention.
- Handle the practical problems: positions, strings, comments, Unicode.
- Decide between a generator and a hand-written lexer.

1 From regular expressions to a table

```

REGULAR EXPRESSION      [a-zA-Z_][a-zA-Z0-9_]*
| THOMPSON'S CONSTRUCTION  O(n) states
| concatenation -> join with epsilon
| alternation  -> new start branching to both
| Kleene star  -> loop back with epsilons
v
NFA (epsilon edges; many states active at once)
| SUBSET CONSTRUCTION
| each DFA state = a SET of NFA states
| worst case 2^n states
v
DFA (one active state, one transition per character)
| HOPCROFT MINIMISATION    O(n log n)
v
MINIMAL DFA -> transition table

Scanning: one array lookup per character. Linear, no backtracking.

```

The value of doing this derivation once is that it is entirely mechanical. A lexer generator is not doing anything you could not do by hand; it is automating three well-defined constructions. **And the end state explains why lexing is never the bottleneck:** the whole token specification collapses into a two-dimensional table indexed by the current state and the next character, so scanning costs one array lookup per input byte.

THE EXPONENTIAL BLOWUP IS REAL AND RARELY MATTERS

Subset construction can produce 2^n DFA states from an n -state NFA, and patterns that achieve the bound are easy to construct — something like $(a|b)^*a(a|b)^k$ does it.

Real token specifications do not come anywhere near it. Identifiers, integer and float literals, operators, and keywords produce minimal DFAs with tens of states. The blowup requires patterns that nobody writes to describe a token.

The pathological behaviour people have actually met comes from general regex engines with backreferences and lookahead — **features that are not regular at all**, which is why those engines backtrack rather than building a DFA, and why catastrophic backtracking exists as a denial-of-service vector.

A lexer's regular expressions are genuinely regular, so the DFA approach always applies and always runs in time linear in the input with no backtracking whatsoever. **That is a guarantee a general-purpose regex library cannot give you**, and it is a good illustration that restricting a language buys properties.

2 The two conventions

MAXIMAL MUNCH AND RULE PRIORITY

Maximal munch: always take the longest match. Given the input `<=`, the lexer produces a single less-than-or-equal token rather than `<` followed by `=`. Implemented by running the DFA until it can no longer advance and backing up to the last accepting state seen.

Rule priority: when two rules match the same longest string, the one listed first wins. The input if matches both the keyword rule and the identifier rule at the same length; keywords are listed first, so if is a keyword.

Together these make lexing fully deterministic with no lookahead beyond the one-character backup the DFA already performs, and they are the reason a lexer never needs to consult the parser.

And maximal munch does occasionally bite. In C++ before 2011, `vector<vector<int>>` was a syntax error because `>>` lexed as the right-shift operator — **a purely lexical rule defeating an obvious syntactic intent**. C++11 fixed it with an explicit special case in the standard, which is the usual resolution: the convention is right almost always, and the exceptions get hard-coded.

Problem	Why it is hard	How it is handled
Source positions	Every later phase needs them for diagnostics , and the lexer is the only component that naturally knows them.	Track from the first character — see the callout below.
Nested comments	Not a regular language. Matching <code>/*</code> with <code>*/</code> to arbitrary depth requires counting.	Special-case it with an explicit depth counter outside the DFA.

Problem	Why it is hard	How it is handled
String escapes	The token's <i>value</i> differs from its source text.	The lexer produces the decoded value alongside the original span, so diagnostics can still quote the source.
Significant indentation	Requires a stack of indentation levels , which is not regular either.	Emit synthetic INDENT and DEDENT tokens so the parser sees an ordinary bracketed structure. Python does exactly this.
Unicode identifiers	Normalisation forms, and visually confusable characters (a Cyrillic <i>a</i> in an identifier).	Adopt a published profile — UAX-31 — and state which normalisation you apply. Do not invent a rule.
Raw strings and interpolation	The lexer's mode changes mid-construct , and interpolation nests arbitrarily.	A mode stack. This is the clearest case where a pure DFA is insufficient.

Several of these are not regular languages at all, which is worth noticing: the clean theory of §1 does not survive contact with a real language specification. **Real lexers are a DFA plus a small amount of explicit state** — a comment depth, an indentation stack, a mode stack — and that is normal rather than a compromise.

CARRY SOURCE POSITIONS FROM THE FIRST CHARACTER

Every error message produced by every later phase needs a source position, and the lexer is the only part of the compiler that sees the raw text.

Retrofitting positions is miserable work. Every AST node, every symbol table entry, every IR value, and every diagnostic has to be threaded through, and the places that lost the information are discovered one error message at a time.

So attach a span — a start and end offset — to every token from the beginning, and propagate it onto every tree node and, where it is affordable, onto IR values too. Debug information (Module 10) depends on the same chain.

Store byte offsets plus a line table rather than line and column per token. It is smaller, it is cheaper to produce, and converting an offset to a line and column when an error is actually reported is a binary search over the line table — which happens rarely and costs nothing.

3 Generator or hand-written

	Generator (lex, flex, re2c)	Hand-written
Effort	The specification is short and declarative — a few dozen lines of patterns and actions.	A few hundred lines of straightforward code.
Speed	Table-driven and very fast.	Often faster — a switch on the first character branch-predicts better than an indirect table lookup, and the common cases can be special-cased.
Error messages	Generic and hard to customise. 'Unexpected character' is about the ceiling.	Exactly what you want — unterminated string literals, suggestions, and context all become easy.
Lexer modes	Supported through start conditions, awkwardly.	Trivial — it is just a variable.
Debugging	Through generated code , which is machine-written and unpleasant to step through.	Ordinary debugging of ordinary code.
Who uses it	Many teaching compilers, and some shipping ones.	GCC, Clang, Rust, Go, V8, and essentially every production compiler.

The last row is the argument, and it surprises people who learned the subject from a textbook where the generator is presented as the obvious choice. **Production compilers hand-write their lexers for diagnostics and modes, not for speed** — the error-message quality that users actually experience is largely decided here, and a generator caps it.

- **Derive the NFA and DFA by hand once**, for five or six token classes. The construction is mechanical and worth having performed rather than only read.
- **Then hand-write the real lexer**: a switch on the first character, dispatching to a small helper per token class.
- **Keep the token type small and trivially copyable** — a kind tag, a span, and an index into an interned string table. Tokens are passed around constantly and a fat token type is felt everywhere.
- **Intern identifiers immediately**. Every identifier becomes a small integer, so comparison is an integer comparison and hashing is free. **This is the cheap decision with the largest downstream effect** — symbol table lookup (Module 05) and type checking (Module 06) both get faster and simpler for nothing.
- **Write the lexer's tests as round-trips**: re-emitting the token stream as text should reproduce the source, modulo whitespace. It catches dropped characters and wrong spans together, with no expected output to maintain.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nystrom — Crafting Interpreters, 'Scanning' (free)	A complete hand-written lexer, explained line by line. The §3 recommendation, implemented.
Stanford CS143 — lexical analysis lectures (free)	Thompson's construction, subset construction, and minimisation, derived rather than asserted.
Russ Cox — Regular Expression Matching Can Be Simple And Fast (free)	The §1 blowup discussion, properly explained , including why general regex engines backtrack and lexers do not.
Unicode Consortium — UAX #31, Identifier and Pattern Syntax (free)	The identifier profile of §2's table. Short, and it saves inventing a rule badly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Convert $(a|b)^*abb$ to an NFA by Thompson's construction, by hand.
- 2 Convert that NFA to a DFA by subset construction, and minimise it.
- 3 **Construct a regular expression whose DFA is exponentially larger** than its NFA, and measure the state count.
- 4 Implement the subset construction and run it on your project language's token set. Report the state count.
- 5 Hand-write a lexer for your language, with spans on every token.
- 6 **Implement maximal munch and demonstrate the C++ >> problem** in a small grammar.
- 7 Add nested comment support and explain why the DFA alone cannot do it.
- 8 **Implement INDENT/DEDENT tokens** for an indentation-sensitive toy language.
- 9 Add string interpolation with a mode stack.
- 10 Write round-trip tests and interning, then measure the effect of interning on a later phase.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the three constructions from regex to minimal DFA.
- 2 Why does a lexer never backtrack, and why do regex libraries?
- 3 State maximal munch and rule priority, and give a case where maximal munch is wrong.
- 4 Name six practical lexing problems and say which are not regular.
- 5 Why track byte offsets rather than line and column?

- 6 Compare generated and hand-written lexers on six axes.
- 7 Why do production compilers hand-write lexers?
- 8 What does interning identifiers buy later?